```
  while not queue.empty()
do                                                    #8
    v ←
queue.dequeue()
       #9
    if isGoal(v)
then
#10
      return (v, parents)
    for e in graph.adjacencyList[v]
do                                          #11
      u ← e.dest
      if distances[u] < inf
then                                                  #12
        distances[u] ← distances[v] +
1                                         #13
        parents[u] ← v
        queue.enqueue(u)
   return (null,
parents)
#14
```

#1 Method `bfs` takes a graph, a starting point vertex, and a predicate (`isGoal`) that takes a vertex and returns `true` if the goal of the search is reached. Method `bfs` returns a pair with the goal vertex, and a dictionary encoding the shortest paths.

#2 Initialize a simple FIFO queue.

#3 Add the starting point to the queue, so that it will be extracted on the first iteration.

#4 Create a new hash table to keep track of the distance of every vertex from vertex `start`, i.e. the minimum number of edges that needs to be traversed to get from vertex `start` to each other vertex.